CS24301 — THEORY (3.0 CR) | MODULE I NOTES | BIT MESRA | NEP SCHEME

Module 1: Lexical Analysis

Language Processors, Buffering, Thompson NFA & Direct DFA

Module I: Lexical Analysis & Automata Theory — Complete Syllabus Topics

1. Language Processors (Compiler, Interpreter, Assembler, Preprocessor)
2. 6 Phases of Compiler Architecture with End-to-End Running Example
3. Compiler Front-End vs Back-End & Pass Structures
4. Lexical Analysis Role, Tokens, Patterns & Lexemes
5. Input Buffering (Two-Buffer Scheme & Sentinels)
6. Regular Expressions & Regular Definitions for Programming Tokens
7. Thompson's Construction Algorithm (RE to NFA Step-by-Step)
8. Subset Construction Algorithm (NFA to DFA Transition Table)
9. Hopcroft's DFA State Minimization Algorithm (Equivalence Partitioning)
10. Lex & Flex Lexical Analyzer Generator Specifications

Topic 1 & 3: Language Processing Systems & Compiler Architecture

A **Compiler** is a sophisticated system software that translates a computer program written in a high-level source language (e.g., C, C++, Java) into an equivalent target program in low-level machine code or assembly language, while detecting and reporting diagnostic errors.

Figure 1.1: Context of a Compiler in Language Processing Pipeline

System Tool	Input → Output	Primary Responsibilities & Core Functions
1. Preprocessor	Raw Source Code → Pure Source Code	Macro expansion (`#define`), file inclusion (`#include`), conditional compilation (`#ifdef`), stripping comments.
2. Compiler	Pure Source Code → Target Assembly	Translates high-level syntax into assembly language, performs type checking, syntax validation, intermediate code generation, and optimization.
3. Assembler	Assembly Code → Relocatable Object Code	Translates mnemonic assembly instructions into binary machine opcodes and resolves memory offsets.
4. Linker / Loader	Object Files + Libs → Absolute Executable	**Linker:** Resolves external cross-file symbol references and library routines. **Loader:** Loads executable binary into primary RAM, sets up stack/heap memory spaces, and initializes CPU instruction pointer.

Topic 2: The 6 Phases of a Compiler (End-to-End Walkthrough)

A compiler operates as a sequence of distinct analytical and synthesis phases. Every phase transforms the source program from one intermediate representation to another, interacting continuously with the **Symbol Table** and **Error Handler**.

🏛️ Complete Running Example: Step-by-Step Translation of Statement `position = initial + rate * 60`

1. **Phase 1: Lexical Analysis (Scanning):**
   Reads raw input character stream and converts into a stream of tokens:

$$\langle \mathbf{id}, 1 \rangle \ \langle = \rangle \ \langle \mathbf{id}, 2 \rangle \ \langle + \rangle \ \langle \mathbf{id}, 3 \rangle \ \langle * \rangle \ \langle \mathbf{num}, 60 \rangle$$

   Symbol table stores: `1: position (float)`, `2: initial (float)`, `3: rate (float)`.

2. **Phase 2: Syntax Analysis (Parsing):**
   Constructs a hierarchical Syntax Tree enforcing operator precedence ($*$ over $+$):

```
      =
     / \
  id1   +
       / \
    id2   *
         / \
      id3  60
```

3. **Phase 3: Semantic Analysis (Type Checking):**
   Checks static semantic rules, ensuring operand type compatibility. Inserts implicit type conversion node $\mathrm{inttofloat}(60)$:

```
      =
     / \
  id1   +
       / \
    id2   *
         / \
      id3  inttofloat(60)
```

4. **Phase 4: Intermediate Code Generation (ICG):**
   Generates machine-independent Three-Address Code (TAC) with at most one operator per instruction:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

5. **Phase 5: Code Optimization:**
   Performs compile-time constant folding and eliminates temporary variables:

```
t1 = id3 * 60.0    -- Constant folded: inttofloat(60) → 60.0
id1 = id2 + t1
```

6. **Phase 6: Code Generation (Target Assembly):**
   Emits target machine instructions using hardware CPU registers (`R1`, `R2`):

```
LDF   R2, id3      ; Load float rate into R2
MULF  R2, #60.0    ; Multiply R2 by float literal 60.0
LDF   R1, id2      ; Load float initial into R1
ADDF  R1, R2       ; Add R2 to R1
STF   id1, R1      ; Store result into memory position
```

Topic 4 & 5: Lexical Analysis & Input Buffering

1. Fundamental Terminology:

Token: An abstract terminal category returned to the parser (e.g., `id`, `num`, `if`, `while`, `assign_op`).
Pattern: A formal grammatical specification (regular expression) describing the set of strings representing a token (e.g., `[a-zA-Z_][a-zA-Z0-9_]*`).
Lexeme: The concrete sequence of source characters matching the pattern (e.g., `counter_variable`, `1024`, `==`).

2. Input Buffering (Two-Buffer Scheme with Sentinels):

Reading one character at a time using direct system calls incurs massive operating system overhead. A **Two-Buffer Scheme** uses two N-byte blocks (typically $N = 4096$ bytes matching disk blocks) managed by two pointers:

`lexemeBegin` Pointer: Marks the beginning of the current token being recognized.
`forward` Pointer: Advances ahead character by character until a token pattern boundary is identified.
Sentinels (`EOF`): Appending an `EOF` marker at the end of each buffer half avoids making two comparisons per character (one for buffer boundary and one for character matching), accelerating lexer throughput by over 200%.

Topic 7: Thompson's Construction Algorithm (RE $\rightarrow$ NFA)

Thompson's Construction converts any Regular Expression r into an equivalent Non-Deterministic Finite Automaton ($N(r)$) with ϵ-transitions in linear time $O(|r|)$:

RE Operation	Thompson's Structural Composition	Key Structural Invariants
1. Basic Symbol (a)	$\text{Start} \xrightarrow{a} \text{Accept}$ (2 states, 1 transition)	Exactly 1 start state and 1 accept state.
2. Concatenation ($r_1 r_2$)	Accept state of $N(r_1)$ merged with start state of $N(r_2)$.	Flows directly from $N(r_1)$ into $N(r_2)$.
3. Union / Alternation ($r_1 \mid r_2$)	New start state branching via ϵ to $N(r_1)$ and $N(r_2)$; both accept states transition via ϵ to a new single accept state.	4 new ϵ-transitions introduced.
4. Kleene Closure (r^*)	New start state branches via ϵ to $N(r)$ and bypasses $N(r)$ to accept state; accept state loops back via ϵ to start of $N(r)$.	Handles zero or multiple repetitions.

Topic 8: Subset Construction Algorithm (NFA $\rightarrow$ DFA)

A Deterministic Finite Automaton (DFA) contains no ϵ-transitions and has at most one outgoing transition per input symbol from any state.

Core Operations in Subset Construction

- ϵ-closure(s): Set of all NFA states reachable from state s taking only ϵ-transitions.
- ϵ-closure(T): $\bigcup_{s \in T}$ ϵ-closure(s) for a set of states T.
- move(T, a): Set of all NFA states reachable from any state in T on input symbol a.

🏛 Step-by-Step Solved Problem: Convert $(a \mid b)^*abb$ to DFA via Subset Construction

NFA States: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. Accept state: $\{10\}$.

Step 1: Compute Initial DFA State $A = \epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$.

Step 2: Compute Transitions for DFA States (D-Tran Table):

DFA State	NFA State Subset	Input 'a'	Input 'b'
A (Start)	{0, 1, 2, 4, 7}	B = {1, 2, 3, 4, 6, 7, 8}	C = {1, 2, 4, 5, 6, 7}
B	{1, 2, 3, 4, 6, 7, 8}	B = {1, 2, 3, 4, 6, 7, 8}	D = {1, 2, 4, 5, 6, 7, 9}
C	{1, 2, 4, 5, 6, 7}	B = {1, 2, 3, 4, 6, 7, 8}	C = {1, 2, 4, 5, 6, 7}
D	{1, 2, 4, 5, 6, 7, 9}	B = {1, 2, 3, 4, 6, 7, 8}	E = {1, 2, 4, 5, 6, 7, 10}
E* (Accept)	{1, 2, 4, 5, 6, 7, 10}	B = {1, 2, 3, 4, 6, 7, 8}	C = {1, 2, 4, 5, 6, 7}

Topic 9: Hopcroft's DFA Minimization Algorithm

Hopcroft's Algorithm finds the unique minimal DFA by iteratively partitioning the set of all DFA states S into disjoint equivalence classes based on behavioral distinguishability:

Initial Partition (P_0): Divide states into two groups: Accept states (F) and Non-Accept states ($S \setminus F$).

$$P_0 = \{F,\ S \setminus F\}$$

Refinement Step: For each group $G \in P$ and each input symbol $a \in \Sigma$, if move(s, a) for states in G leads into different partitions, split G into sub-partitions.
Termination: Repeat refinement until no partition can be further split ($P_{k+1} = P_k$).
State Compression: Replace each final partition group with a single representative state in the minimal DFA.

🏛 Top BIT Mesra Exam Questions & Answers (Module I)

Q1. Explain the role of Lexical Analyzer and why it is separated from the Parser. (8 Marks)

1. **Simplicity of Compiler Design:** Separating lexical scanning from grammatical parsing simplifies both phases. The parser deals with clean token streams rather than raw character-level whitespace, newlines, and comments.
2. **Compiler Efficiency:** Specialized high-speed buffering techniques can be applied to the lexical scanner. Over 70% of compilation time is spent reading source characters.
3. **Portability:** Character-set idiosyncrasies (ASCII, UTF-8, Unicode) are isolated strictly inside the lexical phase, making the parser platform-independent.

Q2. State the difference between Compiler and Interpreter across 5 engineering parameters. (6 Marks)

1. **Translation Mechanism:** Compiler translates entire source code into native machine code before execution; Interpreter translates and executes line-by-line at runtime.
2. **Execution Speed:** Compiled machine code executes 10–50× faster than interpreted bytecode.
3. **Memory Usage:** Compiler requires memory for object code storage; Interpreter requires memory for runtime engine.
4. **Error Reporting:** Compiler reports all syntax/semantic errors collectively during compilation; Interpreter stops at the first runtime error encountered.
5. **Examples:** C, C++, Rust (Compiled); Python, Ruby, PHP (Interpreted).

Q3. Describe the structure and working of Lex/Flex tool. (6 Marks)

A Lex program consists of 3 distinct sections separated by `%%` delimiters:
1. **Definitions Section:** Header includes, declarations, and regular definitions (e.g., `digit [0–9]`).
2. **Rules Section:** Pairs of regular expressions and corresponding C action code (e.g., `{digit}+ { return NUM; }`).
3. **User Subroutines Section:** Auxiliary C functions including `main()` and `yywrap()`.
Lex compiles the specification into a fast transition table-driven DFA scanner named `yylex()`.